Temas Selectos de Física Computacional III: Introducción a la ciencia de datos. Clase 12: 30 de Octubre de 2026

Dr. Jaime Octavio Guerra Pulido

1 Introducción

En esta clase, estudiaremos el modelo neuronal de McCulloch-Pitts. Después, estudiaremos el perceptrón de Rosenblatt y veremos que los perceptrones pueden emplearse siempre para resolver problemas de clasificación utilizando líneas de decisión.

2 Computación neuronal

En la década de 1930, los trabajos de Alan Turing y Emil Post propusieron modelos de **autómatas** capaces de realizar cálculos lógicos y verificar teoremas de lógica de primer orden mediante manipulación simbólica. En particular, el **modelo de Turing** planteó una máquina que, a través de una cinta dividida en celdas con símbolos, era capaz de leer un símbolo, consultar una tabla de reglas, sobrescribir un símbolo (o no hacerlo), cambiar de estado interno y moverse a la izquierda o a la derecha.

Estos modelos formaron la base teórica para el desarrollo posterior de la computación. Ya en 1943, Konrad Zuse construyó una computadora que emulaba el comportamiento de una máquina de Turing. Hoy en día, la mayoría de los lenguajes de programación modernos siguen este paradigma: los programas reciben variables de entrada, las procesan mediante operaciones algebraicas y devuelven un resultado como salida.

Sin embargo, en 1943, Warren McCulloch y Walter Pitts introdujeron un enfoque completamente distinto al de Turing. Inspirados en el funcionamiento de las neuronas biológicas, propusieron un modelo de computación basado en **redes neuronales artificiales (ANN)**. En lugar de manipular símbolos sobre una cinta, como en la máquina de Turing, este nuevo modelo determinaba la *validez de un teorema lógico* mediante la activación de un conjunto de unidades llamadas *neuronas*, conectadas entre sí a través de sinapsis.

En este esquema, la computación no se realiza secuencialmente sobre una cinta, sino en paralelo, mediante una red que propaga señales entre nodos. Las neu-

ronas artificiales se *activan* o *inhiben* en función de las entradas que reciben, lo cual permite representar **procesos lógicos** a través de la dinámica de la red.

3 Neuronas de McCulloch-Pitts

En el modelo de McCulloch-Pitts, los canales de entrada de cada neurona se llaman **dendritas** y el único canal de salida recibe el nombre de **axón**. El axón de la neurona (que puede ramificarse) se conecta a su vez con las dendritas de otras neuronas a través de una **sinapsis**. Los nombres vienen de una neurona biológica en la Figura 1.

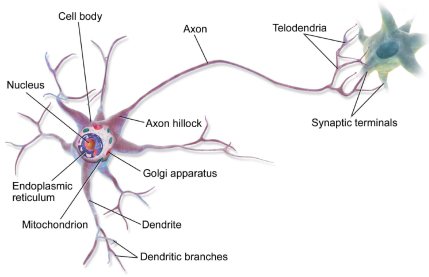


Figure 1: Una neurona biológica.

Para decidir si debe activarse o no, una neurona artificial calcula la suma de las señales que recibe desde otras neuronas. Si una conexión sináptica es **excitadora**, aporta un valor de $+1$; si es **inhibidora**, contribuye con -1. La neurona suma todas estas señales y compara el resultado con un umbral prefijado U.

Si la suma total, denotada como x, supera el umbral, **la neurona se activa**. En ese caso, su salida toma el valor 1. Si no se supera el umbral, permanece inactiva, y su salida es 0. Este comportamiento puede expresarse usando la **función escalón de Heaviside**:

$$f(x) = \begin{cases} 1, & \text{si } x > U \\ 0, & \text{si } x \leq U \end{cases}$$

La salida de una neurona artificial se basa en **dos operaciones matemáticas consecutivas**. La primera consiste en calcular la suma total de excitaciones e inhibiciones que recibe desde otras neuronas. Esta suma se expresa como:

$$x = \sum_i c_i d_i$$

La segunda operación consiste en aplicar la función escalón de Heaviside al valor obtenido. Esto determina si la neurona se activa o no:

$$s = H(x - U)$$

donde:

- x es la suma total de señales recibidas.
- U es el umbral de activación.
- H es la función de Heaviside, definida como:

$$H(x - U) = \begin{cases} 1, & \text{si } x \geq U \\ 0, & \text{si } x < U \end{cases}$$

Este modelo describe el comportamiento más básico de una neurona artificial, en el que se activará únicamente si la señal neta recibida supera el umbral establecido. La representación del modelo se muestra en la Figura 2.

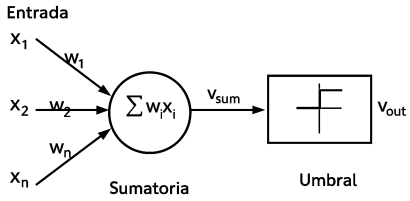


Figure 2: Modelo McCulloch-Pitts.

3.1 Ejemplo de álgebra booleana usando ANNs

Para entender mejor la tarea, consideremos un ejemplo: Jorge lleva un paraguas si está soleado o si está lloviendo. Hay cuatro situaciones dadas, y debemos decidir cuándo Jorge llevará el paraguas. Las situaciones son:

- X_1: ¿Está lloviendo?
- X_2: ¿Está soleado?

Los valores de ambos escenarios pueden ser 0 o 1. Usamos un peso de 1 para X_1 y X_2, y una función de umbral de 1. Por lo tanto, el modelo de red neuronal se verá como en la Figura 3.

- Primera situación: No está lloviendo ni está soleado.
- Segunda situación: No está lloviendo, pero está soleado.

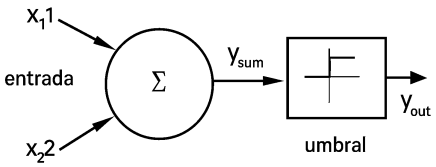


Figure 3: Red neuronal.

- Tercera situación: Está lloviendo, pero no está soleado.
- Cuarta situación: Está lloviendo y también está soleado.

Para analizar las situaciones utilizando el modelo neuronal de McCulloch-Pitts, consideremos las señales de entrada como una tabla de verdad:

Situación	x_1	x_2	y_{sum}	y_{out}
1	0	0	0	0
2	0	1	1	1
3	1	0	1	1
4	1	1	2	1

Podemos expresar matemáticamente las ecuaciones del modelo como:

$$y_{sum} = \sum_{i=1}^{2} w_i x_i$$

$$y_{out} = f(y_{sum}) = \begin{cases} 1 & \text{si } x \geq 1 \\ 0 & \text{si } x < 1 \end{cases}$$

A partir de la tabla de verdad, podemos concluir que en las situaciones donde el valor de y_{out} es 1, Jorge necesita llevar un paraguas. Por lo tanto, **llevará un paraguas en los escenarios 2, 3 y 4**.

3.2 Implementa tu modelo neuronal de McCulloch-Pitts

Podemos implementar el modelo neuronal de McCulloch-Pitts como sigue:

```
import numpy as np

def neurona_mcculloch_pitts(entradas, pesos, umbral):
    # Calcular la suma ponderada de las entradas
```

```
    suma_ponderada = np.dot(entradas, pesos)

    # Aplicar la función de activación (escalón de Heaviside)
    salida = 1.0 if suma_ponderada >= umbral else 0.0
    return salida

# Valores de entrada
entradas = np.array([0.5, 0.3, 0.8])
pesos = np.array([0.2, 0.4, 0.6])
umbral = 0.7

# Calcular la salida de la neurona
salida = neurona_mcculloch_pitts(entradas, pesos, umbral)
print("Salida:", salida)
```

Dadas las siguientes entradas y pesos, la salida de la neurona de McCulloch-Pitts se calcula como:

$$x = \sum_i w_i \cdot x_i = (0.5)(0.2) + (0.3)(0.4) + (0.8)(0.6) = 0.70$$

Como la suma ponderada $x = 0.70$ es igual al umbral $U = 0.7$, entonces la neurona se activa:

$$s = H(x - U) = H(0.70 - 0.70) = H(0) = 1$$

Por tanto, la **salida esperada es 1**.

3.3 Aplicación: puerta lógica AND

Una de las primeras aplicaciones del modelo es resolver algunas puertas lógicas. La tabla de la verdad de la puerta AND es

```
import pandas as pd
columnas = ['x1', 'x2', 'AND']
puerta_and = [[0, 0, 0], [0, 1, 0], [1, 0, 0], [1, 1, 1]]
tabla = pd.DataFrame(puerta_and, columns=columnas)
tabla.head()
```

Una forma simple de representar el comportamiento de la compuerta lógica AND es mediante una **recta de separación** en el plano definido por las variables de entrada x_1 y x_2. Podemos adoptar la siguiente regla de decisión:

$$\text{Salida} = \begin{cases} 1, & \text{si } x_1 + x_2 \geq 2 \\ 0, & \text{si } x_1 + x_2 < 2 \end{cases}$$

Esta expresión define una frontera lineal dada por la recta $x_1 + x_2 = 2$ que separa el espacio de entradas en dos regiones:

- Cuando ambas entradas son 1, se cumple la condición $x_1 + x_2 = 2$, por lo tanto la salida es 1.
- En todos los demás casos, la suma es menor que 2, por lo tanto la salida es 0.

Implementa AND con una neurona de McCulloch-Pitts usando pesos iguales a 1 y un umbral $U = 2$.

```
casos = [
    ([0, 0], 0),
    ([0, 1], 0),
    ([1, 0], 0),
    ([1, 1], 1),
]
pesos = np.array([1, 1])
umbral = 2

for entrada, salida_esperada in casos:
    salida = neurona_mcculloch_pitts(np.array(entrada), pesos, umbral)
    print(f"Entrada: {entrada} → Salida: {int(salida)} (Esperado: {salida_esperada})")
```

4 Modelo de perceptrón de Rosenblatt

El perceptrón de Rosenblatt está basado en el modelo neuronal de McCulloch-Pitts. Su representación se presenta en la Figura 4. El perceptrón recibe un conjunto de entradas $x_1, x_2, \ldots, x_n$. El **combinador lineal** (o sumador) calcula la combinación lineal de las entradas aplicadas a las sinapsis, cuyos pesos sinápticos son $w_1, w_2, \ldots, w_n$. Luego, el **limitador** verifica si la suma resultante es positiva o negativa. Si la entrada al nodo limitador rígido es positiva, la salida es $+1$; si la entrada es negativa, la salida es -1.

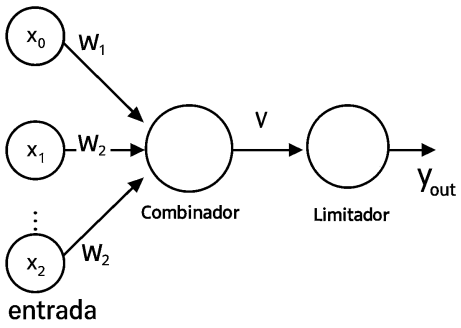


Figure 4: Perceptrón de Rosenblatt.

Matemáticamente, la entrada al limitador se expresa como:

$$v = \sum_{i=1}^{n} w_i x_i$$

Sin embargo, el perceptrón incluye un valor ajustable o sesgo (*bias*) como un peso adicional w_0. Este peso adicional está asociado a una entrada ficticia x_0, que se asigna un valor de 1. Esta consideración modifica la ecuación anterior a:

$$v = \sum_{i=0}^{n} w_i x_i$$

La salida se decide mediante la expresión:

$$y_{\text{out}} = f(v) = \begin{cases} +1, & \text{si } v > 0 \\ -1, & \text{si } v < 0 \end{cases}$$

El objetivo del perceptrón es clasificar un conjunto de entradas en dos clases c_1 y c_2. Esto se puede lograr utilizando una regla de decisión muy simple: asignar las entradas a c_1 si la salida del perceptrón, y_{out}, es $+1$, y a c_2 si y_{out} es -1.

Para un espacio de señales de n-dimensiones (es decir, un espacio para n señales de entrada), la forma más simple del perceptrón tendrá dos regiones de decisión, correspondientes a las dos clases, separadas por un hiperplano definido por:

$$\sum_{i=0}^{n} w_i x_i = 0$$

Por lo tanto, para dos señales de entrada representadas por las variables x_1 y x_2, la frontera de decisión es una línea recta de la forma:

$$w_0 x_0 + w_1 x_1 + w_2 x_2 = 0 \quad \text{o bien:}$$

$$w_0 + w_1 x_1 + w_2 x_2 = 0 \quad [\text{dado que } x_0 = 1]$$

Supongamos un perceptrón con valores de pesos sinápticos w_0, w_1 y w_2 de -2, $\frac{1}{2}$ y $\frac{1}{4}$, respectivamente. La frontera de decisión lineal será de la forma:

$$-2 + \frac{1}{2} x_1 + \frac{1}{4} x_2 = 0$$

Multiplicando por 4 para simplificar, obtenemos:

$$-8 + 2x_1 + x_2 = 0$$

$$2x_1 + x_2 = 8$$

Por lo tanto, cualquier punto (x_1, x_2) que se encuentre por encima de la frontera de decisión (como se muestra en la Figura 4) se asignará a la clase c_1, mientras que los puntos que se encuentren por debajo de la frontera se asignarán a la clase c_2.

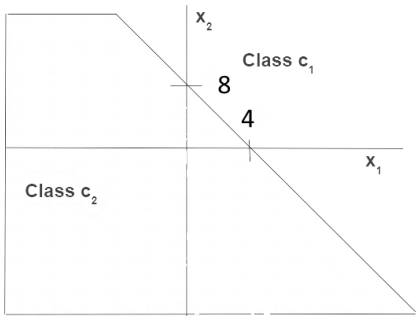


Figure 5: Frontera de decisión.

Vemos que, para un conjunto de datos con **clases linealmente separables**, los perceptrones pueden emplearse siempre para resolver **problemas de clasificación** utilizando líneas de decisión (en espacios de 2 dimensiones), planos de

decisión (en espacios de 3 dimensiones) o hiperplanos de decisión (en espacios de n-dimensiones). Los valores adecuados de los pesos sinápticos pueden obtenerse entrenando el perceptrón.

Sin embargo, hay una suposición importante para que el perceptrón funcione correctamente: las dos clases deben ser linealmente separables, es decir, deben estar suficientemente separadas entre sí. De lo contrario, si las clases no son linealmente separables, el problema de clasificación no puede resolverse utilizando un perceptrón.

4.1 Aplicación del perceptrón: puerta lógica AND

A continuación se muestra cómo entrenar una neurona perceptrón para aprender una función lógica (AND) utilizando `Scikit-learn`.

```
from sklearn.linear_model import Perceptron
X = tabla.values[:, 0:2]
y = tabla.values[:, 2]
nn = Perceptron(eta0=0.01, max_iter=10, random_state=1)
nn.fit(X, y)
print(nn.coef_) # pesos
print(nn.intercept_) # sesgo
```

El perceptrón se llama con los valores`eta0=0.01`: tasa de aprendizaje, y `max_iter=10`: número máximo de épocas.

Después del entrenamiento, se obtienen los siguientes parámetros:

- Pesos aprendidos:

$$\texttt{nn.coef_} = [0.02,\ 0.02] \quad \Rightarrow \quad w_1 = 0.02,\ w_2 = 0.02$$

- Sesgo (bias):

$$\texttt{nn.intercept_} = -0.03 \quad \Rightarrow \quad b = -0.03$$

El modelo evalúa la condición:

$$y = \begin{cases} 1 & \text{si } 0.02x_1 + 0.02x_2 - 0.03 \geq 0 \\ 0 & \text{si no} \end{cases}$$

Esta expresión define una recta de decisión en el plano de las entradas:

$$0.02x_1 + 0.02x_2 = 0.03 \quad \Rightarrow \quad x_1 + x_2 = 1.5$$

Por tanto, el modelo predice `1` solamente cuando la suma de las entradas es al menos 1.5, lo cual es coherente con el comportamiento de una compuerta lógica **AND**, checa la predicción:

```
from sklearn.metrics import accuracy_score
y_pred = nn.predict(X)
print("Precisión:", accuracy_score(y, y_pred))
print("Salidas predichas:", y_pred)
```

Ahora grafica la región de decisión para el perceptrón entrenado

```
from mlxtend.plotting import plot_decision_regions
import matplotlib.pyplot as plt
plot_decision_regions(X, y, clf=nn)
plt.show()
```

Porque con solo 4 puntos discretos, el gráfico de `plot_decision_regions` no puede representar bien la línea. Imprime la frontera real aprendida por el perceptrón:

```
print("w1 =", nn.coef_[0][0])
print("w2 =", nn.coef_[0][1])
print("b =", nn.intercept_[0])
print("Recta: x1 + x2 =", round(-nn.intercept_[0]/nn.coef_[0][0], 2))
```

4.2 Limitaciones del perceptrón

Un perceptrón funciona de manera muy exitosa con conjuntos de datos que poseen patrones linealmente separables. Sin embargo, en situaciones prácticas, eso es una condición ideal difícil de alcanzar. Este fue precisamente el punto señalado por Minsky y Papert en su trabajo de 1969. Ellos demostraron que un perceptrón básico no es capaz de aprender a calcular ni siquiera un XOR de 2 bits. Ahora explicaremos la razón detrás de este hecho. Consideremos la tabla de verdad que muestra la salida de una función XOR de 2 bits:

```
cols = ['x1', 'x2', 'XOR']
puerta_xor=[[0,0,0], [1,0,1], [0,1,1], [1,1,0]]
df = pd.DataFrame(puerta_xor, columns=cols)
df.head()
```

Como podemos observar en la Figura 6, los datos no son linealmente separables. Solo una frontera de decisión curva puede separar correctamente las clases. Para abordar este problema, la otra opción es usar dos líneas de frontera de decisión en lugar de una. Esta es la filosofía que se utiliza para diseñar el modelo de perceptrón multicapa (*multi-layer perceptron* o MLP).

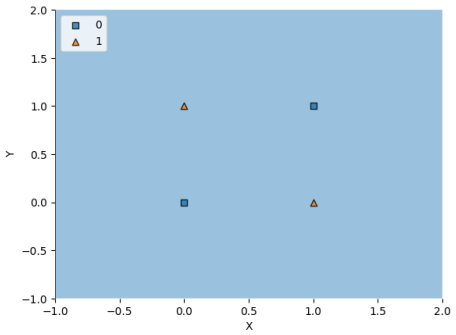


Figure 6: Frontera de decisión en el caso XOR.